Unified ETH experience for hardware signers

Threats overview

Delivery	**Payload**
Phishing	Drainer
Hacked hosting	Redirect to malicious contract (same ABI)
Hijacked DNS	**Full compromise**
Supply chain attack	

(and install an EDR if you manage third party funds)

USE

A

HARDWARE

WALLET

Clear signing ?

Display as much understandable information as possible about a transaction

Using the minimum number of third parties

Clear signing a TX for nerds, calldata edition

Data present : cannot read TX

01020304…….08 : worse, useless

0102030405060708 : TX hard to understand without secondary host/display

0x617ba037 : can read the descriptor and fields

0x6440f144b7e50d6a8439336510312d2f54beb01d

10000000000000000000

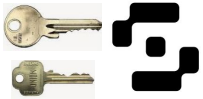
Supply : knowledge of the ABI 🥗

Asset: 0x6440f144b7e50d6a8439336510312d2f54beb01d

Amount: 10000000000000000000

Clear signing a TX for users

Supply 10 BOLD to AAVE : read the TX intent



Supply 10 BOLD to AAVE : read the TX final intent (governance)

Approve
Supply

Supply 10 BOLD to AAVE : read the TX final intent (complex)

- 10 BOLD : balance change : TX simulation, different thing

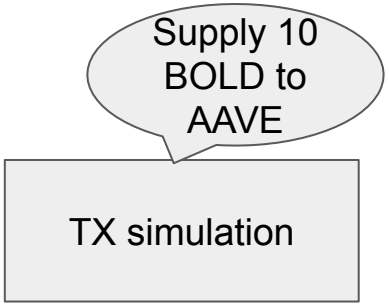
+ 10 AAVEBOLD

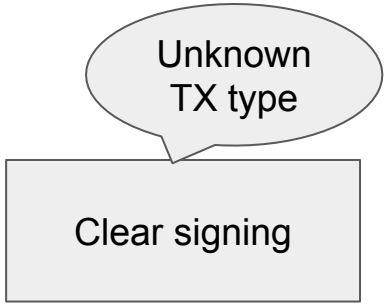
TX simulation

Multiple issues

- Dependant on the current state of the network (TOSTOB)
- “Red pill attacks”
- Privacy

Use it as an additional layer, flash warning if significant difference in outcome





Clear signing for messages

“Sign in to https://www.very.legit” : read a message string (EIP-191)

```
  "SafeTx": [
    { "type": "address", "name": "to" },
    { "type": "uint256", "name": "value" },
    { "type": "bytes", "name": "data" },
    { "type": "uint8", "name": "operation" },
    { "type": "uint256", "name": "safeTxGas" },
    { "type": "uint256", "name": "baseGas" },
    { "type": "uint256", "name": "gasPrice" },
    { "type": "address", "name": "gasToken" },
    { "type": "address", "name": "refundReceiver" },
    { "type": "uint256", "name": "nonce" }
  ]
}
```

: read a typed message (EIP-712)

```
  "SafeTx": [
    { "type": "address", "name": "to" },
    { "type": "uint256", "name": "value" },
    { "type": "bytes", "name": "data" },
```

: handle as a transaction 🥗

Missing context

ABI / EIP-712 message description is not sufficient

```
"type": "function",
"name": "supply",
"inputs": [
  { "name": "asset", "type": "address", "internalType": "address" },
  { "name": "amount", "type": "uint256", "internalType": "uint256" },
  { "name": "onBehalfOf", "type": "address", "internalType": "address" },
  { "name": "referralCode", "type": "uint16", "internalType": "uint16" }
],
```

Is a token

```
"SafeTx": [
  { "type": "address", "name": "to" },
  { "type": "uint256", "name": "value" },
  { "type": "bytes", "name": "data" },
  { "type": "uint8", "name": "operation" },
  { "type": "uint256", "name": "safeTxGas" },
  { "type": "uint256", "name": "baseGas" },
  { "type": "uint256", "name": "gasPrice" },
  { "type": "address", "name": "gasToken" },
  { "type": "address", "name": "refundReceiver" },
  { "type": "uint256", "name": "nonce" }
]
```

Is a transaction

CROPS troubles

Censorship resistance

- Curated lists / Registry : signature to be verified

Onchain solutions

- Complex to backport (add a new proxy contract)
- Complex to verify for hardware signers

Good overview from Kalapaja

Complexity for hardware signers

MCU based, typically low RAM (top 128 Kb, usually /2 /4 on a smartcard)

Limited screen size

- Bad UX means users are always clicking yes (decades of PKI experience)

Different communication expectations

- Airgap (QR, EIP-4527) getting popular, good design target

Not exactly a recent problem

Tech | 07/15/2021

ETHEREUM SMART CONTRACT PLUGINS

A new frontier for the security of human interactions with smart contracts

Past solutions

Vendor specific lists of ABIs

NatSpec

On-chain description (EIP-719, ERC-4430/6384)

Application specific calldata parsing code

Needs a sandbox

Calldata decoding without ABIs ?

How to avoid XKCD #927 ?

Stagnant | Standards Track: ERC

ERC-4430: Described Transactions

A technique for contracts to provide a human-readable description of a transaction's side-effects.

Authors: Richard Moore (@ricmoo), Nick Johnson (@arachnid)

Created: 2021-11-07

Stagnant | Standards Track: ERC

ERC-6384: Human-readable offline signatures

A method for retrieving a human-readable description of EIP-712 typed and structured data.

Authors: Tal Be'ery <tal@zengo.com>, RoiV (@DeVaz1)

Created: 2023-01-08

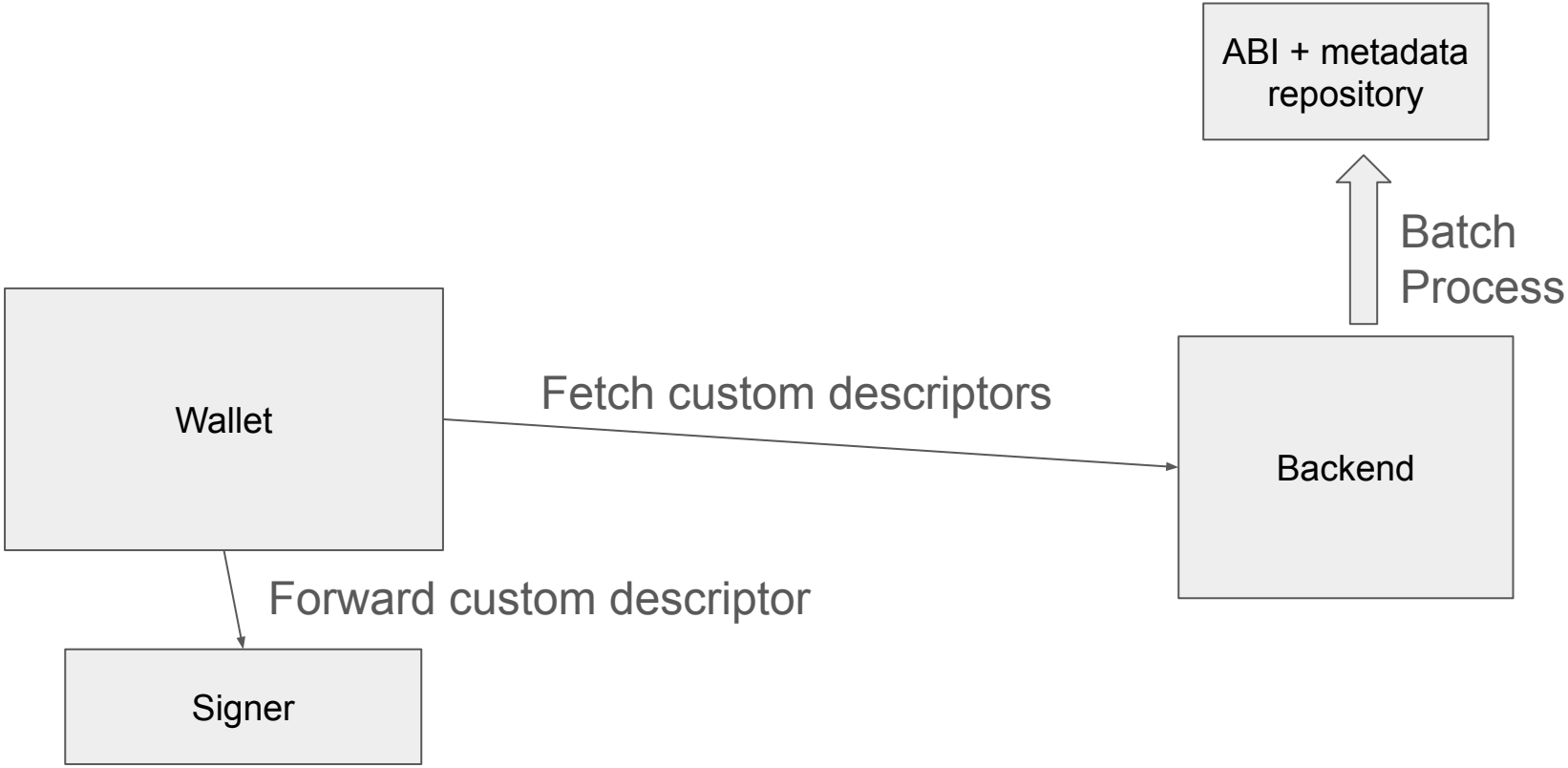
Trustless Signing UI Protocol #719

Closed

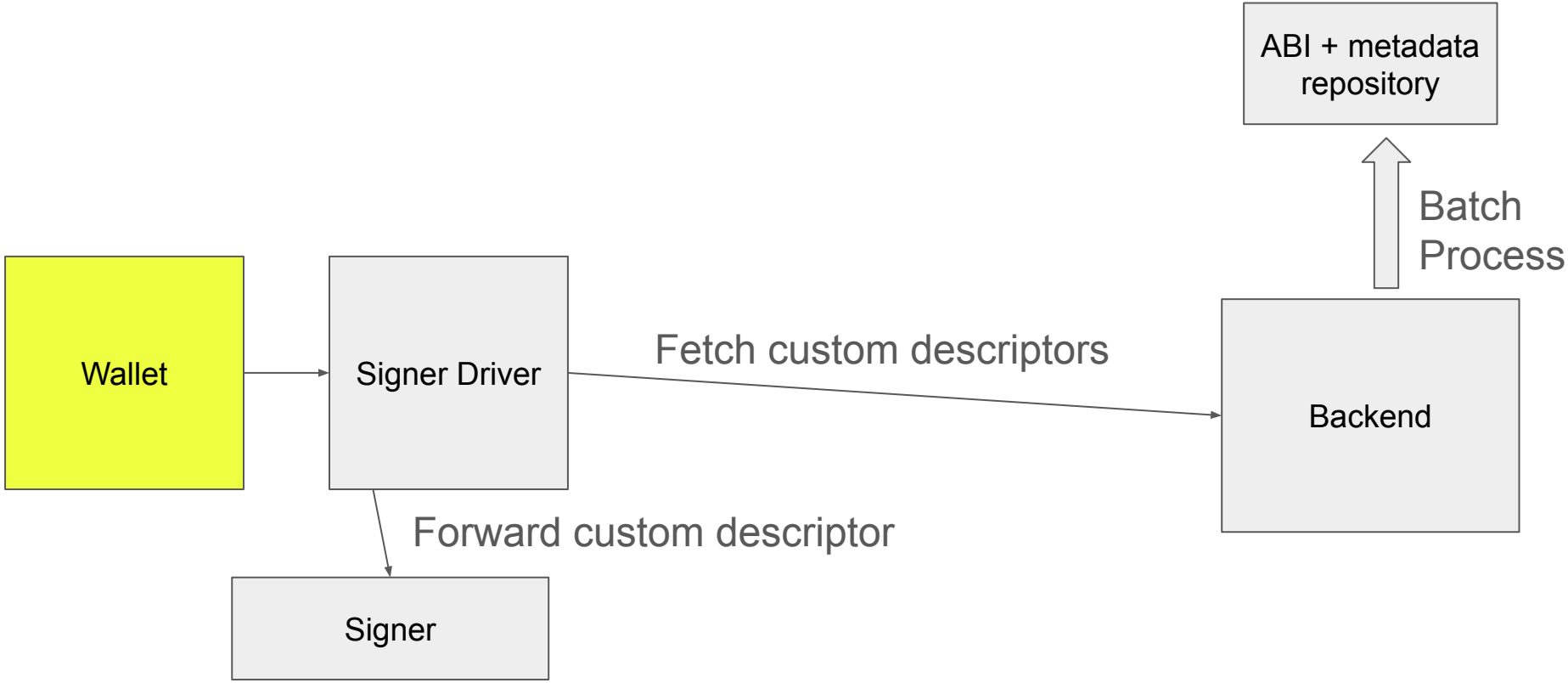
MicahZoltu opened on Sep 22, 2017 · edited by MicahZoltu

A lot of debate has been happening around how we can make signing a more secure process for end users. One common arguments is that while all of the proposed solutions may resolve the problem for advanced users (deve savvy users, professionals, etc.) it doesn't address the problem of naive users not understanding what they are sig can often be thought of as the legalese presented to people in contracts: it is better than signing a blank sheet of to the average person its mostly gibberish.

Typical architecture today



External signer architecture today



Problems with this architecture

Signer driver needs to access remote data

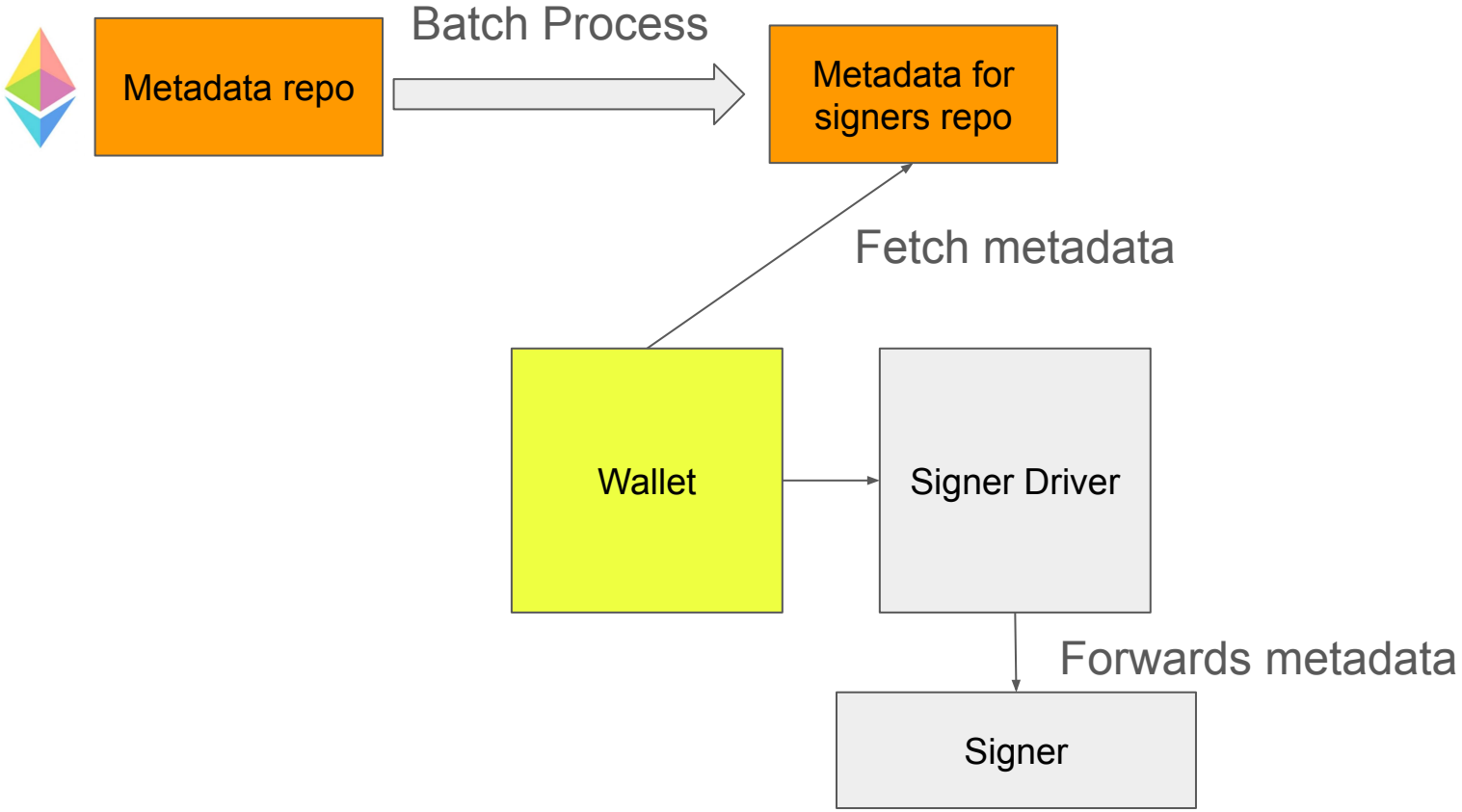
- Security issue (already happened, 🦊 would have been saved by LavaMoat, 🐰 not so much)
- Privacy issue (precise TX data shared with signer infrastructure)

Signer might not have a remote infrastructure

- DIY projects
- Airgapped signers (QR)

Descriptor format is not standardized

Target external signer architecture



ERC-7730 one pager

Richer metadata around smart contract ABI and EIP-712

- More descriptive intents (descriptor level and optionally higher level)
- Fields to hide
- Fields to group
- Specific fields types : token, calldata, …

No code execution on signer involved and compatible with legacy contracts

Covers both display format and provenance

ERC-7730 calls to action

Need more descriptors for widely used contracts

Signer wire format is being defined

Feedbacks for on-chain registry 🥗

Mechanism to handle more complex cases

- Multiple batching logic (such as Safe multisend)
- Uniswap router

Zero Knowledge Proofs to the rescue

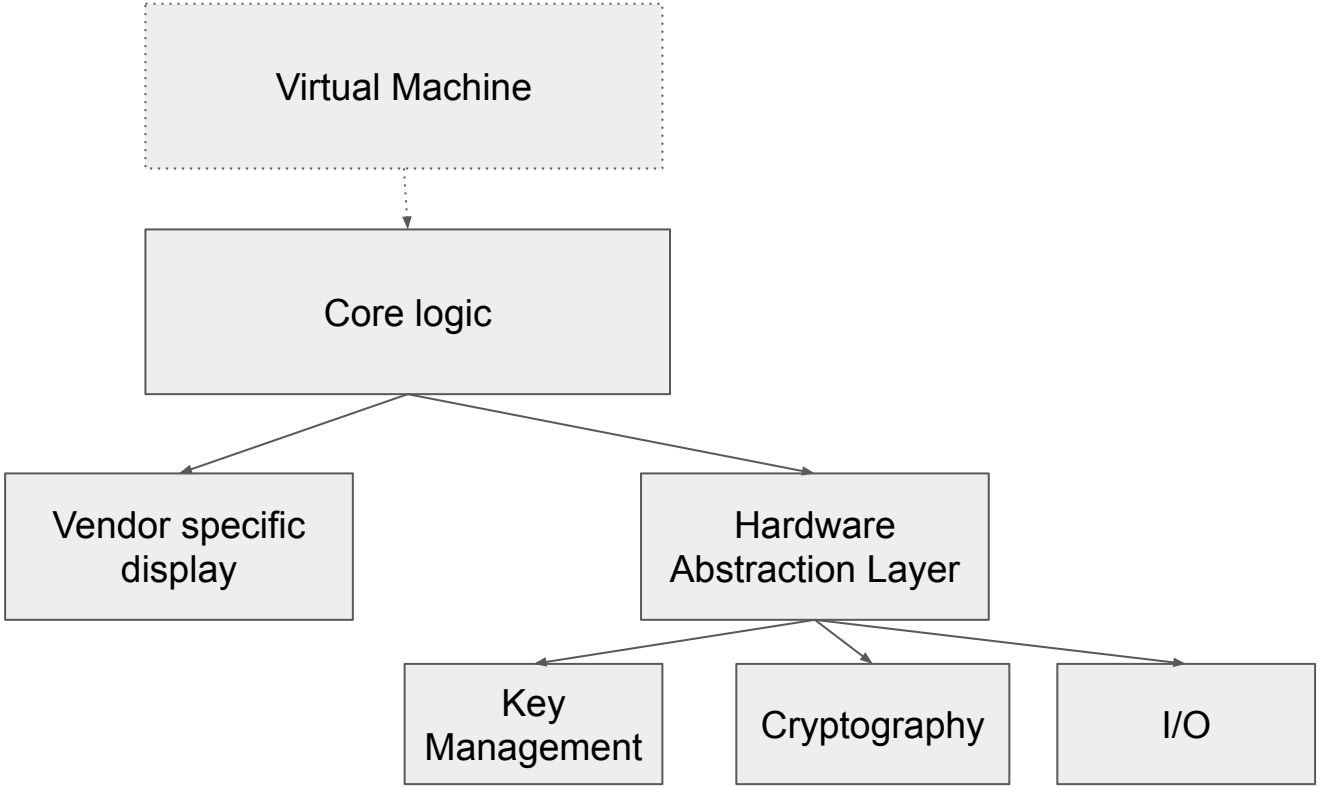
Build a ZKP of the expected transaction action, verify it on hardware 🥗

```
                    tx, readable, π
┌──────────┐ ──────────────────────────────────────► ┌──────────────────┐
│  Prover  │                                          │  Hardware Wallet │
│  (dapp / │                                          │  secure element  │
│ relayer) │                                          └──────────────────┘
└──────────┘                                                   │
     │ tx, readable (witness)                        ③ receives tx, readable
     ▼                                                         │
┌──────────────────┐                                  ④ recomputes locally
│    ZK Circuit    │                                    h(tx)  from raw bytes
│  Groth16 BLS12-381 │                                  h(readable)  from string
│                  │                                          │
│  h(tx) = hash(tx)  │                                  ⑤ verifies π against
│  h(rbl) = hash(rbl)│ ──► public signals ──────────────  h(tx) and h(readable)
│  readable = ABI(tx)│         h(tx)                            │
│                  │         h(readable)              valid  → display readable
└──────────────────┘                                  invalid → reject
     │
     └──► proof π (π.A 96B | π.B 192B | π.C 96B)
```

Universal signer application features

- Portability
  - Across vendors and communication mechanisms (airgap good target)
- Clear signing
  - Support for developers and users
- Emergent privacy needs
  - Privacy protocols key management
  - Clear signing for privacy protocols
- Designed from scratch with CROPS in mind

Universal signer application architecture



Why use a Virtual Machine ?

Optional

- Lack of RAM (securely shared with the host)
- Lack of application management (only done by Ledger and Passport Prime)

Inspired from Ledger Vanadium

RISC-V CPU, hardened against glitch attacks

Standardizing a wire format

CBOR already used in EIP-4527, non verbose binary format, extensible

Suitable for other transports

CDDL for Eth Sign Request.

The following specification is written in Concise Data Definition Language `CDDL`. UUIDs in this specification notated UUID are CBOR binary strings tagged with #6.37, per the IANA `CBOR Tags Registry`.

```
; Metadata for the signing request for Ethereum.
;
sign-data-type = {
    type: int .default 1 transaction data; the unsigned data type
}

eth-transaction-data = 1; legacy transaction rlp encoding of unsigned transaction data
eth-typed-data = 2; EIP-712 typed signing data
eth-raw-bytes=3;   for signing message usage, like EIP-191 personal_sign data
eth-typed-transaction=4; EIP-2718 typed transaction of unsigned transaction data

; Metadata for the signing request for Ethereum.
; request-id: the identifier for this signing request.
; sign-data: the unsigned data
; data-type: see sign-data-type definition
; chain-id: chain id definition see https://github.com/ethereum-lists/chains for detail
; derivation-path: the key path of the private key to sign the data
; address: Ethereum address of the signing type for verification purposes which is optional

eth-sign-request = (
    sign-data: sign-data-bytes, ; sign-data is the data to be signed by offline signer, currently it can be unsigned tran
    data-type: #3.401(sign-data-type),
    chain-id: int .default 1,
    derivation-path: #5.304(crypto-keypath), ;the key path for signing this request
    ?request-id: uuid, ; the uuid for this signing request
    ?address: eth-address-bytes,            ;verification purpose for the address of the signing key
    ?origin: text  ;the origin of this sign request, like wallet name
)
request-id = 1
sign-data = 2
data-type = 3
chain-id = 4 ;it will be the chain id of ethereum related blockchain
derivation-path = 5
address = 6
origin = 7
eth-address-bytes = bytes .size 20
sign-data-bytes = bytes ; for unsigned transactions it will be the rlp encoding for unsigned transaction data and ERC 712
```

Embedded development strategy in 2026

Extensive use of automatic programming

Specification with higher end frontier model

Choose the implementation language later (first C, target Rust)

Adversarial testing plan - test attacks in the message flow and format

Code review with different models

Rinse, repeat

Boomer AI setup for design & code

Frontier model : Claude Opus 4.6

Future tests : GPT-5.4, GLM 5.1, Kat-Coder-Pro V2

Local models (likely outdated in a couple weeks/months)

- Qwen3.5-27B : specification/test review
- Qwen3.5-122B : development

Used mostly with Claude Code

Claude, design an EIP 712 embedded signer

```
struct Mail {
    address from;
    address to;
    string contents;
}
```

Definition of `encodeType`

The type of a struct is encoded as `name ‖ "(" ‖ member₁ ‖ "," ‖ member₂ ‖ "," ‖ … ‖ memberₙ ")"` where each member is written as `type ‖ " " ‖ name`. For example, the above `Mail` struct is encoded as `Mail(address from,address to,string contents)`.

If the struct type references other struct types (and these in turn reference even more struct types), then the set of referenced struct types is collected, sorted by name and appended to the encoding. An example encoding is `Transaction(Person from,Person to,Asset tx)Asset(address token,uint256 amount)Person(address wallet,string name)`.

Claude, design an EIP 712 embedded signer

4.2 PROVIDE_TYPE (CMD 0x01)

Send the EIP-712 type encoding string for the current struct, e.g. `"Mail(address from,Person to,string contents)Person(string name,address wallet)"` . May require multiple APDUs for long strings.

4.3 STRUCT_START (CMD 0x02)

Begin a struct hash scope. Must be preceded by PROVIDE_TYPE for this struct.

Field	Value
P1	Struct role: 0x00 = EIP712Domain, 0x01 = primary message, 0x02 = nested struct, 0x03 = array element struct
P2	Number of fields in this struct (1–255)
Data	(optional) UTF-8 struct type name for display, e.g. "Person"

Device action:

- Push a new scope onto the scope stack: save current `slot_base` and `slot_index` and `field_count` .
- Allocate `field_count + 1` slots (1 typeHash + N fields) starting at the current free position in the flat encoding buffer.
- Slot 0 of this scope should already contain the typeHash from PROVIDE_TYPE.
- Set expected field counter to P2.

```
── Phase 1: Domain separator ──────────────────────────────────────────────
2  PROVIDE_TYPE        0x00  0x01  "EIP712Domain(string name,uint256 chainId)"
3  STRUCT_START        0x00  0x02  "EIP712Domain"
4  FIELD (string)      0x70  0x01  [4,"name","Ether Mail"]
      → device hashes "Ether Mail", stores 32-byte digest
      → displays "name: Ether Mail", waits for user OK
5  FIELD (uint256)     0x22  0x01  [7,"chainId",<32-byte BE encoding of 1>]
      → device stores 32-byte value directly
      → displays "chainId: 1", waits for user OK
6  STRUCT_END          0x00  0x00  (none)
      → device hashes: typeHash||slot[1]||slot[2] → domainSeparator
```

Claude, design an EIP 712 embedded signer

4.3 STRUCT_START (CMD 0x02)

Begin a struct hash scope. Must be preceded by PROVIDE_TYPE for this struct.

Field	Value
P1	Struct role: 0x00 = EIP712Domain, 0x01 = primary message, 0x02 = nested struct, 0x03 = array element struct
P2	Number of fields in this struct (1–255)
Data	`[1 byte type_name_len][type_name_len bytes struct_type_name][1 byte field_name_len][field_name_len bytes parent_field_name]` — field_name is the name of the field in the **parent** struct that holds this nested struct. For top-level structs (P1=0x00 or 0x01), field_name_len is 0.

TOCTOU verification (for nested structs, P1=0x02 or 0x03):

1. Compute `fnv1a_32(struct_type_name)` and compare against `parent.expected_fields[field_index].type_hash`. Mismatch → SW=0x6A80.
2. Compute `fnv1a_32(parent_field_name)` and compare against `parent.expected_fields[field_index].name_hash`. Mismatch → SW=0x6A80.
3. Verify `parent.expected_fields[field_index].kind == STRUCT`. Mismatch → SW=0x6A82.

Device action:

- Push a new scope onto the scope stack: save current `slot_base` and `slot_index` and `field_count`.
- Allocate `field_count + 1` slots (1 typeHash + N fields) starting at the current free position in the flat encoding buffer.
- Slot 0 of this scope should already contain the typeHash from PROVIDE_TYPE.
- Set expected field counter to P2.

there is a TOCTOU vulnerability between the type encoding struct of a struct and its fields - a correct type can be advertised in the definition then be different when the field is passed.

fnvla_32

Claude, design an EIP 712 embedded signer

```
2  DEFINE_TYPE         0x01  0x00  0x02  "EIP712Domain"
3  DEFINE_FIELD        0x02  0x00  0x00  [6,"string",4,"name"]
4  DEFINE_FIELD        0x02  0x00  0x00  [7,"uint256",7,"chainId"]
      → device stores: type_defs[0] = {name="EIP712Domain", field_count=2}
        field_defs[0] = {p1=0x70, kind=FIELD, name="name"}
        field_defs[1] = {p1=0x22, kind=FIELD, name="chainId"}
```

```
12  STRUCT_START        0x03  0x00  0x00  (none)
      → P1=0x00 (domain), looks up "EIP712Domain" in type_defs
      → typeHash not yet computed → computes it now:
        hash_init()
        feeds "EIP712Domain(string name,uint256 chainId)"
             (type names derived from stored field_defs[0..1].p1)
        hash_finalize() → typeHash, cached in type_defs[0].type_hash
      → pushes scope, stores typeHash in encoding slot[0]

13  FIELD               0x04  0x70  0x01  [4,"name","Ether Mail"]
      → verifies: field_defs[0].kind==FIELD ✓
                  field_defs[0].expected_p1==0x70 ✓
                  "name" == field_defs[0].field_name ✓
      → device hashes "Ether Mail" → stores digest in slot[1]
      → displays "name: Ether Mail", waits for user OK

14  FIELD               0x04  0x22  0x01  [7,"chainId",<32-byte BE of 1>]
      → verifies against field_defs[1] ✓
      → stores 32-byte value in slot[2]
      → displays "chainId: 1", waits for user OK

15  STRUCT_END          0x05  0x00  0x00  (none)
      → verifies 2 fields received == 2 expected
      → hash_init(), feeds slot[0]||slot[1]||slot[2], hash_finalize()
      → stores result as domainSeparator
```

using fnv1a_32 is not great as an attacker could easily find a collision and again create a TOCTOU vulnerability. Can you think of a scheme that keeps the RAM usage in check while preventing this vulnerability fully ?

Thank you
@btchip
(when my account isn’t suspended)